

Recipe-Dependent Rankings: Method Orderings in Language-Model Screens Are Properties of the Measurement, Not the Methods

Bharath V.

Independent researcher

bharath.vbcr@gmail.com

2026-08-24

Short-horizon architecture and optimizer comparisons are the default currency of empirical language-model research. We report a controlled attempt to replicate one such comparison — our own — and show that the headline result was a property of the measurement recipe rather than of the architectures being compared.

A single-seed run on an RTX 3070 Ti at batch size 8 placed the point where softmax attention overtakes minGRU between **6.6M and 7.4M** training tokens on a 12-layer / 768-dim model. Re-running the same matched pair on an NVIDIA GH200 at **n = 5 seeds** through a 50M-token horizon does not reproduce that location. Instead the mean validation-loss curves cross **twice**: minGRU overtakes attention at **1.05M** tokens and attention overtakes minGRU for good at **12.35M** tokens, holding a **-0.227** [-0.244, -0.211] advantage at 50M. Within the original 6.6–8.2M window minGRU leads by **0.13–0.18** nats on every seed.

We then isolate the confounds one at a time, each with five seeds. Truncating the run to 20M tokens while letting the cosine schedule follow the truncated step count moves the late crossing from 12.34M to **14.58M**; restoring the 50M cosine horizon and stopping at the same 20M budget recovers **12.34M** (per-seed 11.93–12.58M) and reproduces the 19.7M gap to within 0.001. Holding batch size at the original 8 on the new hardware produces **no crossing at all** through 7.38M tokens on any seed. A matched-batch 50M ranking of ten mixer configurations places attention first at **4.222** [4.204, 4.240] but statistically tied with a 10×minGRU + 2×attention hybrid at **4.232** [4.210, 4.254].

We observe the same phenomenon on an independent axis. In a five-stage optimizer funnel on Apple silicon, MONA ranks first at 16M parameters and **last of four** at the `exact_128m` configuration (128,367,988 parameters) — “exact” there names the funnel’s bit-exactness resume gate, recorded in `research/exact-128m-gate-polar.json`, not a round parameter count; the eventual 128M champion, Polar Express Muon, ranked third in the initial learning-rate screen and loses to Muon NS5 when run back down at small scale. We also correct the confidence intervals originally computed for that funnel: with the correct two-seed Student-*t* multiplier, the champion-selection intervals **overlap**, and the selection rests on a sign-consistent 2-of-2 ordering rather than on separated intervals.

The transferable claim is not “attention wins late.” It is that **the crossing token is a function of batch size, learning-rate horizon, hardware and evaluation protocol**, and that a ranking measured at one recipe carries no license to be reported as a property of the architectures.

One limitation is large enough to state in the abstract. Every run here uses standard parametriza-

tion with a single global Muon learning rate, so we cannot yet separate “the ranking depends on the recipe” from “the arms were unequally well tuned at each recipe.” §8 states that competing explanation, gives the evidence that partially bounds it, and specifies the μP arm — a 2×2 over parametrization and architecture on the suite 24 recipe, plus two cheaper ablations — that would settle it, with a pre-registered reading of each possible outcome.

1 1. Introduction

A large fraction of practical architecture and optimizer selection happens on short runs. The reasoning is economic: a screen that costs 1% of a full run and preserves the ordering of candidates is worth more than the full run. The assumption doing the work is that the ordering is preserved.

That assumption has been challenged before, along several independent axes, and it has usually lost. Melis et al. re-evaluated a sequence of LSTM successors under large-scale black-box tuning and found that properly regularized standard LSTMs outperformed the newer architectures that had been reported to beat them [6]. Musgrave et al. audited four years of deep metric learning and found the accumulated gains to be marginal once experimental methodology was controlled [38]. Henderson et al. showed that non-determinism and intrinsic variance in deep reinforcement learning make published comparisons difficult to interpret without seed discipline and significance reporting [8]. Dodge et al. made the dependence explicit: test-set scores alone are insufficient for model comparison, and several published comparisons **invert** when the computation budget spent on hyperparameter search changes [7]. Hooker’s *hardware lottery* names the systems version of the same problem — an idea can win because it suits the available hardware and software rather than because it is better [26].

The specific variable this paper is about — **training horizon** — is less often treated as a ranking confound, even though the pieces needed to predict that it would be are individually well established. Optimal learning rate depends strongly on token horizon, with longer training requiring smaller learning rates and the optimal value following its own scaling law [31]. The number of steps required to reach a target loss varies enormously with batch size and workload, and much of the historical disagreement about whether batch size affects final quality is attributable to differences in metaparameter tuning at different batch sizes [27]. The largest useful batch size is itself a moving target predicted by the gradient noise scale, which changes *over the course of a run* as loss decreases [28], and critical batch size scales primarily with data size rather than model size [29]. Small batches are not merely a scaled-down version of large ones: they require different optimizer hyperparameters, and holding Adam’s second-moment decay fixed rather than its token half-life mis-specifies them [30]. Compute-optimal scaling work established that conclusions drawn at fixed data are systematically misleading [9, 10], and Choshen et al. found enough seed-level variability in scaling-law estimation that training several small models is sometimes more informative than training one large one [37].

Put together, these say that batch size, learning-rate schedule and horizon jointly determine where a training curve is at any given token count. What they do not say — and what we could not find measured directly — is what that does to the **ordering** of two architectures whose curves cross, and specifically to the *token at which they cross*, which is the quantity that short-run comparisons implicitly report.

This paper reports what happened when we tested that against a result we had previously published ourselves.

The original observation. In June 2026 we ran three 12-layer / 768-dim mixers — minGRU, softmax attention, and Mamba-2 [1, 2] — on FineWeb-Edu [25] for 8.192M tokens on an RTX 3070 Ti Laptop (batch size 8, sequence length 512, Muon, one seed). minGRU led at every early checkpoint; attention closed the gap monotonically and overtook between 6.6M and 7.4M tokens (suite 14). Two follow-up pairs at a smaller model and a longer horizon preserved the *shape* — early recurrent lead, late attention overtake — with the flip landing near 8.19M and 7.37M tokens respectively (suite 15). We described this as “bias wins early, capacity wins late” and treated the ~7M token location as the finding.

That interpretation was not unreasonable in isolation. It is consistent with the broader picture in which pure state-space and recurrent stacks match Transformers on many tasks but lag on those requiring copying or in-context retrieval [5], and with the general expectation that an architecture’s inductive bias should matter most when data is scarce. What made it a mistake was not the story. It was that we reported a **number** — the crossing token — from a single seed on a single recipe, and treated it as a property of the architectures.

What this paper adds. Access to an NVIDIA GH200 made a five-seed replication affordable. We ran the matched attention/minGRU pair plus eight additional mixer configurations to 50M tokens, then ran three targeted isolates that each change exactly one factor. The specific contributions:

1. **A failed location replication with a measured alternative.** At $n = 5$ on the GH200 the 6.6–7.4M crossing does not appear; two crossings appear instead, at 1.05M and 12.35M (§4.2). Every seed agrees on the sign at every marker except the 12.3M tie.
2. **Learning-rate horizon is a first-order confound for the crossing token, not only for final loss.** Truncating the token budget without holding the cosine horizon fixed moves the late crossing by 2.248M tokens — 18% of its location — and the effect is measurable in the learning rate itself, which at the crossing marker is ~47% of its long-horizon value (§4.3). This is the ranking-level consequence of the horizon-dependence of optimal LR reported in [31].
3. **Batch size, not just hardware, changes the early ordering.** At batch size 32 attention leads the first evaluation on all five seeds; at batch size 8 on the *same* GPU minGRU leads the first evaluation on all five seeds and never surrenders it through 7.38M tokens (§4.4). Both batch sizes are far below any plausible critical batch size at this scale [29], so this is not a large-batch degradation effect.
4. **A matched-batch 50M mixer ranking** in which the previously reported ordering of hybrids is substantially rearranged once batch and evaluation count are held fixed (§4.5), and in which the best hybrid is statistically indistinguishable from pure attention.
5. **An independent replication of the phenomenon on the optimizer axis**, including a statistical correction to our own previously reported selection intervals (§5). Zhao et al. found that most modern optimizers perform comparably at their own optimal hyperparameters [36]; our funnel is a case study in what happens when the tuning budget is too short to find those hyperparameters.
6. **A protocol** (§9) and an explicit list of claims we withdraw (§7.3).
7. **A named, designed follow-up** (§8): the single most likely explanation we did *not* control for is parametrization, and we specify the μP arm that would test it.

We deliberately do not claim a mechanism. “Recurrent inductive bias helps early, attention capacity wins late” remains a plausible story that fits these curves; it is not established by them, and it does not predict the crossing token, which is the quantity that actually moved.

2 Background and related work

Sequence mixers. minGRU is a minimal gated recurrent unit whose recurrence is parallelizable over sequence length, proposed as evidence that heavily simplified RNNs remain competitive with Transformers at moderate scale [1]. Mamba-2 recasts selective state-space models and attention variants inside a shared structured-semiseparable-matrix framework, yielding a core layer 2–8 $\times$ faster than Mamba while remaining competitive on language modeling [2]. Gated DeltaNet combines gated adaptive memory control with the delta update rule to improve retrieval-limited behavior in linear Transformers [3]. Multi-head Latent Attention compresses the KV cache into a latent vector and was introduced with DeepSeek-V2 [4]. All four, plus attention and six hybrid interleavings, are implemented in the `nanolab` codebase used here.

Hybrids. Interleaving a small number of attention layers into an otherwise recurrent or state-space stack is now a standard recipe. At 8B parameters and 3.5T tokens, a 43% Mamba-2 / 7% attention / 50% MLP hybrid exceeded a matched Transformer on all twelve standard tasks evaluated, while pure SSMS lagged on copying and in-context-learning tasks [5]. Our §4.5 board is a small-scale, matched-recipe version of the same question, and reaches a compatible conclusion: the best hybrid is statistically indistinguishable from pure attention, and pure recurrent stacks are clearly behind.

Benchmark validity and budget-dependent conclusions. The methodological literature summarized in §1 [6, 7, 8, 38] establishes that measured rankings depend on the resources spent measuring them. That work is almost always framed around *hyperparameter search* budget or implementation variance. The closest precedent to our framing is Dodge et al.’s expected validation performance as a function of budget, which makes the budget an explicit axis of the reported result rather than an implicit constant [7]. We adopt the same posture for the training horizon: if two curves cross, the crossing token is part of the result and must be reported with the recipe that produced it.

Horizon, batch size and schedule. Optimal learning rate depends on the token horizon, with longer runs requiring smaller learning rates and the optimal value obeying its own scaling law that permits transfer from short horizons to long [31]. This is the mechanism behind §4.3: when a run is truncated and the cosine schedule follows the truncated step count, the two runs are on materially different learning rates at every shared token marker, and we measure the ratio at $\sim 47\%$ at the crossing point. On the batch axis, Shallue et al. characterized steps-to-target as a function of batch size across 35 workloads and attributed much of the literature’s disagreement about batch-size effects on quality to differences in metaparameter tuning [27]; McCandlish et al. introduced the gradient noise scale as a predictor of the largest useful batch size and noted that it *increases as loss decreases within a run* [28]; Zhang et al. measured how critical batch size scales in pre-training from 85M to 1.2B parameters and found it scales primarily with data size rather than model size [29]. Marek et al. revisited very small batches and proposed holding Adam’s second-moment half-life fixed in tokens rather than holding its decay rate fixed, finding small batches stable and more robust to hyperparameter choice than conventional wisdom holds [30]. Our batch-8 and batch-32 recipes (§4.4) sit well below any critical batch size at this scale, which is why we read the batch effect there as an ordering effect rather than a large-batch quality penalty.

Parametrization and hyperparameter transfer. μ P makes many optimal hyperparameters approximately invariant to width, enabling a tune-small / transfer-large workflow [12]; Depth- μ P extends the analysis to depthwise transfer and identifies fundamental limitations when each resid-

ual block is itself deep, as in modern Transformers [33]; u- μ P combines μ P with unit scaling so that default values are near-optimal and FP8 training works out of the box [34]. The picture is not settled in μ P’s favor: Everett et al. trained tens of thousands of models across four parameterizations, three optimizers and fourteen model sizes to 26.8B parameters, and found that *all* parameterizations can achieve hyperparameter transfer, with a per-layer learning-rate prescription for standard parametrization outperforming μ P [32]. Kalra and Barkeshli localized μ P’s apparent advantage over standard parametrization under AdamW to a single factor — maximizing the embedding-layer learning rate, which in standard parametrization acts as a bottleneck that induces instability [35]. Our runs use standard parametrization with a global Muon learning rate and no per-layer prescription; §8 is about what that leaves untested.

Scale as a confound. Compute-optimal scaling established that model size and token count must be scaled together and that conclusions at fixed data mislead [9, 10]. Wortsman et al. reproduced large-scale instabilities in small models at high learning rates and studied how warm-up, weight decay and μ Param change loss sensitivity to learning rate [11] — the same class of interaction we measure at the ranking level in §4.3. Choshen et al. derived best practices for estimating scaling laws from 485 pretrained models and over 1000 fitted laws, finding that fitting to intermediate checkpoints substantially improves accuracy and that seed variability can make several small models more useful than one large one [37]. Schaeffer et al. argued that apparently sharp scale-dependent transitions can be artifacts of the chosen metric [13]; our crossings are measured on a continuous metric (held-out cross-entropy), which removes that particular explanation but not the recipe dependence.

Optimizers. The optimizer arm of this work compares Muon-family matrix-orthogonalizing methods against full-parameter baselines. Muon has been shown to scale to large LLM training with weight decay and per-parameter update-scale adjustment, reporting roughly $2\times$ computational efficiency versus AdamW under compute-optimal training [14]. Polar Express replaces Newton–Schulz with a minimax-optimal polynomial iteration for the polar decomposition and reports consistent validation-loss improvements when integrated into Muon on GPT-2-scale FineWeb training [15]. NorMuon adds neuron-wise adaptive second moments on top of orthogonalized updates [16]. MONA adds a curvature-aware Nesterov acceleration term inside Muon’s gradient pipeline and reports gains at 1B–68B MoE scale [17]. Muown treats the row-magnitude vector as an explicit optimizer variable and reports improved perplexity over Muon, SOAP, AdamW and Lion on FineWeb-Edu from 124M to 2.7B [18]. Baselines and controls include AdamW [19], Lion [20], Sophia [21], Schedule-Free AdamW [22], and Prodigy [23]. Two results frame §5. Zhao et al. compared SGD, Adafactor, Adam, Lion and Sophia across model sizes and hyperparameter settings and found that — SGD aside — they perform comparably both at their optima and in robustness to hyperparameter misspecification, suggesting optimizer choice can be driven by memory and implementation considerations [36]. Community benchmarking efforts such as AlgoPerf exist precisely because optimizer comparisons are so sensitive to tuning protocol [24]. If optimizers are largely comparable *when each is properly tuned*, then a short screen that cannot tune them is measuring something else — which is what §5.4 reports.

Data. All runs in this paper train on FineWeb-Edu, the educational subset of FineWeb [25].

3 3. Experimental setup

3.1 3.1 Model and training

The mixer experiments use a fixed 12-layer / 768-dim decoder (“124M shape”) with sequence length 512, trained with Muon at learning rate 6e-4 on FineWeb-Edu, `mixer_chunk = 32`, `compile = False`, GPU-resident data, and `eval_train = False`. The only lever that varies across arms is the layer mixer. Seeds are 1337, 42, 100, 2026, 777 for every five-seed suite.

Hardware for suites 22–26 is a Lambda Cloud GH200 (97,871 MiB HBM, aarch64, PyTorch 2.7.0 + CUDA 12.8). Suites 14–16 ran earlier on an RTX 3070 Ti Laptop (8 GB). `torch.compile` stalled in Inductor for attention on GH200 aarch64, so every job in suites 22–26 ran uncompiled; this is a uniform choice across arms, not a per-arm difference.

Warm-up, evaluation and checkpoint cadences are expressed in **tokens**, not steps, so that a wider microbatch still lands evaluations near the original suite-14 markers (batch $8 \times$ sequence 512 = 4,096 tokens/step; evaluation every 200 such steps, i.e. every 0.8192M tokens).

3.2 3.2 Evaluation and statistics

Validation loss is reported at the nearest logged evaluation to each named token marker, with `eval_iters = 20` for every arm in the matched suites. For a marker with five seeds we report the mean and a 95% Student-*t* interval with four degrees of freedom ($t_4 = 2.776$). The gap is defined as attention - minGRU, so a negative gap means attention is better.

Crossing points are obtained by linear interpolation of the two **mean** curves on attention’s evaluation grid; per-seed crossings are computed the same way on individual curves and reported as a range so that a mean crossing cannot be mistaken for a single-seed artifact.

Three rules were applied throughout and are worth stating because they cost us results:

- **No cross-recipe tables.** Cells measured at batch 8 never appear in the same table as cells measured at batch 32, even when the token markers match.
- **A `best_val` field is not a paired snapshot.** Minimum-over-all-evaluations is not a ranking and is never reported as one.
- **An auto-generated intersection over mixed token grids is not a source of truth.** When arms ran at different batch sizes their eval grids do not align, and a naive intersection produces flat, wrong early markers.

3.3 3.3 Suites

Suite	Question	Hardware / batch	Seeds	Budget
14	Does the 2M-token mixer ranking survive to 8.2M?	3070 Ti / bs8	1	8.192M
15	Does the crossover shape survive a smaller model and a longer run?	3070 Ti / bs16	1	16.4M, 30.3M
22	Does the 6.6–7.4M location replicate at $n = 5$ to 50M?	GH200 / bs32 (+bs96 drift)	5	50M

Suite	Question	Hardware / batch	Seeds	Budget
23	Independent 20M pair, cosine following max_steps	GH200 / bs32	5	20M
24	Same, with the 50M cosine horizon held	GH200 / bs32	5	20M
25	Original batch size 8, on the new hardware	GH200 / bs8	5	8.192M
26	Matched-batch 50M ranking of the eight drifted arms	GH200 / bs32	5	50M

Suites 24, 25 and 26 are *isolates*: each changes exactly one factor relative to suite 22 and verifies the recipe fingerprint (**batch_size**, **eval_iters**, **max_steps**, **lr_max_steps**) on every individual job config before the results are read.

4 4. Results: the mixer axis

4.1 4.1 The original observation (suite 14)

RTX 3070 Ti, batch 8, sequence 512, 2,000 steps = 8.192M tokens, seed 1337, one seed per arm.

Tokens	minGRU	Attention	Mamba-2	gap (A-G)
0.8M	6.334	6.516	6.493	+0.182
4.1M	5.549	5.624	5.769	+0.075
6.6M	5.353	5.358	5.560	+0.005 (tied)
7.4M	5.249	5.239	5.469	-0.010 ← overtakes
8.2M	5.155	5.136	5.383	-0.019

Suite 15 preserved the shape at two other configurations: a 6L/384d model at batch 16 crossed near **8.19M** tokens (step 1000 of 2000), and the full 12L/768d model at batch 16 crossed near **7.37M** tokens (step 900 of 3700). Each of those four arms is a single seed.

The crossing locations above are read from paired evaluation curves. The end-of-run figures for these four arms — 4.852 / 5.021 at 16.4M and 4.260 / 4.575 at 30.3M — are **best_val** fields, i.e. minimum-over-all-evaluations, and by the rule we set out in §3.2 they are **not** paired snapshots and **not** a ranking. An earlier version of this section reported them as though they were, which is exactly the failure §3.2 is meant to prevent. We state them here only as run summaries, and no ordering in this paper depends on them. The same caveat applies to the 8.2M row of the suite-14 table above, which is that run’s **done** record rather than a matched-token evaluation.

This is the entire evidential basis for the “~7M crossover” claim: **one seed** at the headline configuration, and two single-seed pairs whose crossings land at 8.19M and 7.37M — already a 1.1M-token spread that we under-weighted at the time.

4.2 4.2 The $n = 5$ replication (suite 22)

Attention and minGRU share one recipe from step 0: batch 32, `eval_iters` = 20, 50M-token cosine, five seeds. Means are nearest-evaluation; intervals are 95% Student- t on five seeds.

Actual tokens	Attention [95%]	minGRU [95%]	gap A-G [95%]
0.836M	6.775 [6.755, 6.794]	6.831 [6.819, 6.844]	-0.056 [-0.065, -0.048]
4.112M	5.888 [5.863, 5.914]	5.661 [5.637, 5.684]	+0.228 [+0.216, +0.239]
6.570M	5.581 [5.573, 5.589]	5.402 [5.390, 5.414]	+0.179 [+0.169, +0.189]
7.389M	5.504 [5.489, 5.519]	5.350 [5.330, 5.370]	+0.154 [+0.145, +0.162]
8.208M	5.434 [5.405, 5.462]	5.299 [5.267, 5.332]	+0.134 [+0.126, +0.143]
12.304M	5.122 [5.104, 5.141]	5.120 [5.097, 5.144]	+0.002 [-0.009, +0.013] (tie)
19.677M	4.743 [4.733, 4.754]	4.936 [4.924, 4.947]	-0.193 [-0.201, -0.185]
49.988M	4.222 [4.204, 4.240]	4.449 [4.423, 4.476]	-0.227 [-0.244, -0.211]

Interpolated mean-curve crossings: **1.049M** (minGRU overtakes attention) and **12.353M** (attention overtakes minGRU and stays ahead through 50M). Per-seed late crossings are **12.031**, **12.216**, **12.453**, **12.474**, **12.579M** — a 0.55M band, so the mean crossing is not one lucky seed. The sign of the gap agrees across all five seeds at every marker except the 12.3M tie.

Three things follow.

The location did not replicate. Suite 14’s 6.6–7.4M window sits squarely inside the minGRU-lead region on this recipe, with minGRU ahead by 0.13–0.18.

There are two crossings, not one. Attention leads the *first* evaluation (0.836M) on every seed here — the opposite of suite 14, where minGRU led from its first logged point. The early crossing at 1.05M is a phenomenon suite 14 could not have seen: its first evaluation was at 0.82M, essentially on top of the crossing, and it had one seed.

Absolute losses are not comparable across the two boxes. At ~8.2M, attention is 5.434 here against 5.136 on the 3070 Ti — a ~0.3 nat gap. The curves are not vertically interchangeable, and no table in this paper stacks them.

4.3 4.3 The learning-rate horizon is a first-order confound (suites 23 and 24)

Suite 23 asked for an independent 20M-token pair on the same batch and evaluation recipe. It recovered the early crossing exactly — **1.047M**, per-seed 1.03–1.09M — and moved the late one to **14.582M** (per-seed 13.709, 14.490, 14.929, 15.058, 15.193M). At 12.3M, minGRU still led by **+0.051** on every seed; at 19.677M attention led by only **-0.065** [-0.081, -0.050], against suite 22’s -0.193 at the same marker.

The cause is mechanical and was verified in the configs rather than inferred: the cosine schedule followed `max_steps` (1220) instead of the 50M horizon (3051). At the 12.3M marker the 20M-horizon learning rate is approximately **47%** of the 50M-horizon learning rate at the same token count. Two runs that are identical in architecture, data, batch, seed set and evaluation protocol are on materially different learning rates at the token where the ranking flips.

Suite 24 is the corrected prefix: identical to suite 23 except `lr_max_steps` = 3051.

Actual tokens	Attention [95%]	minGRU [95%]	gap A-G [95%]
0.836M	6.778 [6.765, 6.791]	6.831 [6.819, 6.844]	-0.053 [-0.055, -0.051]
4.112M	5.888 [5.863, 5.914]	5.661 [5.637, 5.684]	+0.227 [+0.214, +0.241]
6.570M	5.580 [5.572, 5.589]	5.402 [5.390, 5.414]	+0.178 [+0.168, +0.189]
7.389M	5.504 [5.489, 5.518]	5.350 [5.330, 5.370]	+0.153 [+0.141, +0.166]
8.208M	5.432 [5.401, 5.464]	5.299 [5.267, 5.332]	+0.133 [+0.125, +0.141]
12.304M	5.122 [5.099, 5.144]	5.120 [5.097, 5.144]	+0.001 [-0.010, +0.013] (tie)
19.677M	4.744 [4.734, 4.754]	4.936 [4.924, 4.947]	-0.192 [-0.199, -0.185]

Crossings: **1.038M** and **12.335M**; per-seed late crossings 11.930, 12.287, 12.365, 12.564, 12.579M. This is an independent ten-job draw that lands on suite 22’s crossing to within 0.02M and reproduces the 19.677M gap to within **0.001**. Suite 23’s 14.6M was the schedule, not seed noise.

This is, to us, the most actionable single finding in the paper. **Truncating a run for a cheaper comparison silently retunes the learning rate at every token you are comparing at.** A study that stops a 50M-token recipe at 20M to save compute is not measuring a prefix of that recipe.

4.4 Batch size changes the early ordering (suite 25)

Suite 22 changed GPU and batch size together. Suite 25 holds batch at suite 14’s value of 8 on the GH200, at suite 14’s 8.192M budget, with the cosine horizon matching the run (2000 steps).

Actual tokens	Attention [95%]	minGRU [95%]	gap A-G [95%]
0.823M	6.736 [6.698, 6.774]	6.567 [6.521, 6.614]	+0.168 [+0.160, +0.177]
4.100M	5.834 [5.799, 5.868]	5.721 [5.686, 5.755]	+0.113 [+0.102, +0.124]
6.558M	5.507 [5.474, 5.539]	5.462 [5.420, 5.504]	+0.045 [+0.028, +0.062]
7.377M	5.425 [5.391, 5.459]	5.395 [5.359, 5.431]	+0.030 [+0.017, +0.043]

No crossing occurs in any logged evaluation, on the mean curves or on any individual seed. minGRU leads from the first evaluation on all five seeds and the gap shrinks monotonically — $0.168 \rightarrow 0.113 \rightarrow 0.045 \rightarrow 0.030$ — without changing sign through 7.38M tokens.

This isolates batch size from hardware. At batch 32 on this GPU, attention leads the first evaluation on every seed; at batch 8 on the *same* GPU, minGRU leads the first evaluation on every seed. Suite 25 reproduces suite 14’s *early shape*, which suite 22 does not, while failing to reproduce suite 14’s 6.6–7.4M overtake, which suite 22 also does not.

One honest gap: the final 8.192M `evaluate()` is written to `done.best_val` and not logged as an eval row, so the last **paired** marker here is 7.377M. `best_val` (attention 5.371 [5.330, 5.412], minGRU 5.353 [5.319, 5.387]) is a minimum over all evaluations, not a paired snapshot, and we do not rank on it. Confidence is **High** for “no flip through 7.38M” and **Medium** for any statement about 8.2M.

4.5 4.5 A matched-batch 50M ranking (suite 26)

Suite 22’s ten-arm 50M board mixed batch 32 / `eval_iters` = 20 with batch 96 / `eval_iters` = 4, because the token budget was drained by repacking leftover jobs at a wider microbatch. That makes it a record of what finished, not a bakeoff. Suite 26 reruns the eight drifted arms at batch 32 and `eval_iters` = 20 from step 0, 40 jobs, 40/40 completed.

Rank	Arm	Source	Mean val [95%]	Seed range
1	attention	s22	4.222 [4.204, 4.240]	4.204–4.237
2	hybrid_mingru10_attn2	s26	4.232 [4.210, 4.254]	4.210–4.249
3	hybrid_gdn_periodic	s26	4.290 [4.271, 4.309]	4.271–4.308
4	hybrid_gdn_bookend	s26	4.301 [4.278, 4.324]	4.277–4.319
5	hybrid_gdn10_attn2	s26	4.314 [4.292, 4.336]	4.290–4.331
6	hybrid_mamba10_attn2	s26	4.333 [4.312, 4.353]	4.313–4.353
7	gdn	s26	4.441 [4.419, 4.462]	4.422–4.461
8	mingru	s22	4.449 [4.423, 4.476]	4.430–4.473
9	mamba2	s26	4.596 [4.562, 4.630]	4.572–4.633
10	mla	s26	4.627 [4.604, 4.650]	4.609–4.647

Horizon and a rejected correction (2026-08-24). Every row above is measured at the same evaluation point, **49,987,584 tokens**: the forty `crossover50m_matched32` runs and the `crossover50m` attention and minGRU arms all carry that point. Where this section compares movement against suite 22 below, those suite-22 values are measured at **49,594,368** tokens instead — a 0.8% shorter horizon — because four `crossover50m` arms end on the shorter grid. That difference is smaller than every rank gap discussed, but it is a cross-horizon comparison and is labelled as one.

A later rebuild of this manuscript (§7.4 item 8) replaced row 2 with **4.275 [4.245, 4.305]** and reported its interval as disjoint from attention’s, on the grounds that “the 4.232 figure appears in no surviving run”. **We have restored 4.232 and reject that correction.** The figure is `hybrid_mingru10_attn2` in `crossover50m_matched32`, and recomputing it from those five runs gives 4.2319 [4.2099, 4.2539] at 49,987,584 tokens — the same point as the attention row it is compared against. The replacement 4.275 is the same arm in `crossover50m`, where its seeds end at 49,594,368. The correction therefore swapped a matched-horizon comparison for a mismatched one, in the course of documenting horizon mismatch as a defect.

The genuine weakness of this row is the one we already state: it is a **cross-suite** comparison, because suite 26 never reran attention or minGRU at 50M. That is gap E2, and it caps this board at Medium-High confidence until those ten jobs are run.

Two arms moved substantially relative to the mixed-recipe board, and both are arms that had trained at batch 96: `hybrid_mamba10_attn2` improved from 4.604 to **4.333** (−0.271) and `hybrid_mingru10_attn2` from 4.275 to **4.232**. GDN-family arms were already at batch 32 and moved only at seed scale (e.g. `hybrid_gdn_periodic` 4.280 → 4.290).

The matched conclusions are narrower than the drifted board suggested:

- Attention has the **best mean**, but is **not separable** from a 10×minGRU + 2×attention hybrid at $n = 5$; the intervals overlap.

- Adding two attention layers to a recurrent stack is worth roughly **0.22** nats over the pure recurrent stack (minGRU 4.449 → hybrid 4.232) and roughly **0.26** over pure Mamba-2 (4.596 → 4.333). This is the largest single effect on the board.
- Pure Mamba-2 and MLA are last, clearly separated from everything above them.
- Nothing here is a throughput claim. Quality ranking and tokens-per-second are separate outcomes and are never combined in this paper.

Confidence is **High** for the eight rerun arms and **Medium-High** for the combined ranking, because the attention and minGRU cells are suite 22’s sample rather than a fresh 50M draw.

4.6 4.6 Summary: what moved the crossing

Every row below is $n = 5$ except the first.

Configuration	Suite	Early crossing	Late crossing
RTX 3070 Ti, bs8, 8.192M budget, 1 seed	14	not observed (minGRU leads eval 1)	6.6–7.4M
GH200, bs32, 50M cosine, 50M budget	22	1.049M	12.353M
GH200, bs32, 50M cosine, 20M budget (independent draw)	24	1.038M	12.335M
GH200, bs32, 20M cosine , 20M budget	23	1.047M	14.582M
GH200, bs8 , own cosine, 8.192M budget	25	not observed (minGRU leads eval 1)	none through 7.38M

Reading down the table: the early crossing is stable at ~1.04–1.05M across every batch-32 recipe and absent at batch 8. The late crossing moves by 2.248M tokens on a change to the learning-rate horizon alone, and does not occur at all within the original budget at the original batch size. The one quantity that was reported as the finding — the token of the flip — is the one quantity that is not stable.

5 5. The same phenomenon on the optimizer axis

The mixer study varied architecture with everything else fixed. The obvious next axis is the optimizer, and it is a well-posed target precisely because optimizer papers are so often introduced on short curves.

5.1 5.1 Funnel design

Sixteen candidates, one engine, one equal-token protocol, on an Apple M5 Pro. Fourteen are implemented natively in Rust and Metal with parity fixtures: Muon NS5, Muon NS3, Polar Express Muon, NorMuon, Muown, MONA, AdamW, Cautious AdamW, Lion, Cautious Lion, momentum SGD, Sophia, Schedule-Free AdamW, and Prodigy.

Two are recorded as **systems-blocked exclusions** rather than dropped: MiMuon (`mimuon_adamw`) requires exact singular-gap routing, which requires a per-matrix SVD; SOAP (`soap_adamw`)

requires a periodic symmetric eigendecomposition. Metal/MPS exposes neither, and an Accelerate fallback would force a host synchronization on every refresh, converting the arm into a measurement of the fallback. They are not silent substitutions and they are **not evidence about those methods**.

The funnel is five stages on an atomic, resumable job ledger, written down before the runs:

1. Five-point logarithmic learning-rate sweep at 16M parameters, 100 steps, seed 1337.
2. Every stable LR winner for 500 steps, seed 1337.
3. The best of those, plus mandatory AdamW and Muon NS5 anchors, for 1,000 steps at seeds 42 and 2026.
4. The top four at the `exact_128m` configuration (128,367,988 parameters) for 500 steps, seed 1337.
5. The top two at 128M for 1,000 steps, seeds 42 and 2026.

Selection is by mean equal-token validation bits-per-byte, with time-to-loss, memory footprint and step time as tie-breakers *only* when intervals overlap.

5.2 The reversal

Stage 3 — 16M parameters, 1,000 steps, two seeds. We report both seeds and two intervals: the $\pm 1.96 \cdot \text{SE}$ interval originally computed, and the correct Student- t interval for $n = 2$ ($t_1 = 12.706$).

Candidate	Seed 42	Seed 2026	Mean BPB	$\pm 1.96 \cdot \text{SE}$	95% t_1 interval
MONA	2.183040	2.178756	2.180898	0.004198	[2.1537, 2.2081]
Polar Muon	2.179047	2.185596	2.182322	0.006418	[2.1407, 2.2239]
NorMuon	2.236222	2.228217	2.232220	0.007845	[2.1814, 2.2831]
Muon NS5	2.225931	2.239343	2.232637	0.013144	[2.1474, 2.3178]
Muon NS3	2.262745	2.255586	2.259166	0.007016	[2.2137, 2.3046]
Muown	2.583694	2.585490	2.584592	0.001760	[2.5732, 2.5960]
AdamW (control)	11.378829	11.179040	11.278935	0.195790	[10.010, 12.548]

The same four candidates at the `exact_128m` configuration (128,367,988 parameters), 500 steps, seed 1337 ($n = 1$, no interval):

Candidate	Val BPB
Polar Muon	2.420958
NorMuon	2.567474
Muon NS5	2.601096
MONA	2.950696

MONA goes from first to last, by more than half a bit per byte, on the same data, the same schedule and the same tuned learning rate that won at 16M.

Stage 5, 128M, 1,000 steps, seeds 42 and 2026:

Candidate	Seed 42	Seed 2026	Mean BPB	95% t_1 interval
Polar Muon	2.167282	2.172555	2.169919	[2.1364, 2.2034]
NorMuon	2.202106	2.200184	2.201145	[2.1889, 2.2134]

Polar Express Muon is the champion. The locked winner passed an exact checkpoint-and-replay gate: loss delta 1.43e-6, gradient-norm delta 0.0, sampled master-weight max delta 2.56e-6, 1,707 dispatches, 13,483 MB resident, on a model of exactly **128,367,988** parameters, with no swap pressure and no recorded failures.

5.3 5.3 A correction to our own statistics

Our earlier write-up of this funnel stated that at stage 5 “the intervals do not overlap, so the champion is decided without ever reaching for a systems tie-breaker.” That was computed with a normal ($z = 1.96$) multiplier on a two-seed standard error. With the correct $t_1 = 12.706$ multiplier, Polar’s interval [2.1364, 2.2034] and NorMuon’s [2.1889, 2.2134] **do overlap**.

What survives is weaker but still real: **both** Polar seeds (2.167282, 2.172555) are below **both** NorMuon seeds (2.200184, 2.202106), a sign-consistent 2-of-2 ordering with a mean separation of 0.031 BPB. The correct statement is that the selection rests on a consistent ordering at $n = 2$, not on separated 95% intervals. The same correction applies at stage 3, where under t_1 intervals the top four candidates are **mutually inseparable** — which makes the subsequent 128M reversal less surprising and more damning of the screen.

The defect was in the funnel tooling, not only in the write-up, and it had teeth: the interval also gated `confidence_interval_overlaps_best`, which decides whether the declared systems tie-breakers (time-to-loss, footprint, step time) are invoked at all. A single-seed arm was additionally recorded with a **zero-width** interval — infinite precision for an arm measured once — so it could be declared separated from every rival. `nanolab/native_funnel.py` now uses a Student-t multiplier with $df = n - 1$, reports an **undefined** (infinite) interval at $n = 1$, and **skips** the systems tie-breakers entirely when any arm has fewer than three seeds, recording `confidence_interval_overlaps_best = None` rather than `False` so that a check which could not run is not recorded as a check that ran. Stored rankings were recomputed with the original values preserved under `ci95_legacy_z`. The recorded champion is unchanged. Full detail in `docs/ISSUES_AND_GAPS_2026-08-22.md` §2.3.

5.4 5.4 The control that failed loudly

AdamW appears at 11.28 BPB in the stage-3 table. It is worth explaining rather than hiding.

At the 100-step LR sweep, AdamW at lr 0.0012 finished at 3.593993 — behind the five Muon-family arms (3.353–3.376) but ahead of Muown (3.642), Sophia (3.643), Cautious Lion (3.739), momentum SGD (3.754), Lion (3.755) and Schedule-Free AdamW (3.778). It looked like a healthy control. At 500 steps it was 8.489102. At 1,000 steps across two seeds it was 11.278935.

Nothing broke: no NaN, no divergence, finite telemetry throughout. The learning rate that looked best over 100 steps simply does not survive 1,000, and short-horizon EMA lag hid that completely at the point where the decision was made. The narrow, transferable version: **a 100-step learning-rate selection is not a learning-rate selection**. It is a measurement of which method has the least EMA lag at 100 steps.

This does not overturn independent evidence that AdamW is quality-competitive. A separate one-seed, 4.096M-token, width-768 bake-off on the 3070 Ti ranked Lion first at 5.557 and AdamW second at 5.565, ahead of Muon at 5.643 — which is exactly why AdamW is a *mandatory* anchor in this funnel rather than an optional arm. In the same bake-off Prodigy soft-diverged to 20.16 and Schedule-Free AdamW plateaued at 7.918 at their defaults. In the native funnel Prodigy is a recorded numerical exclusion rather than a missing result: two of its five learning-rate sweep points terminated non-finite after two logged steps, its best finite screen result was 11.889982 BPB at lr

0.25 — an order of magnitude behind every other candidate — and its stage-2 500-step job at that same learning rate also terminated non-finite.

5.5 The mirror image: downward transfer fails too

If the story were “small-scale rankings underestimate the eventual winner,” it would be a tidy lesson and half true. We ran the 128M champion back down against Muon NS5 on the small **sota** model, 3,000 steps, two seeds, at matched throughput on the quality-oriented flash path:

Seed	NS5	Polar	Δ
1337	2.122381	2.127708	+0.005327
42	2.127647	2.131005	+0.003358
mean	2.125014	2.129357	+0.004342

The 128M champion loses at small scale, on both seeds. The champion’s tuned learning rate of 0.05 applied at the small scale was worse still. So the ranking is not a biased-but-monotone estimate of the large-scale ranking; it is **a property of the scale**, with no safe direction of extrapolation. In this funnel the eventual champion was third at the LR sweep and second at 1,000 steps — a shortlist of two would have discarded it at stage 1 had the LR screen been the only cut.

6 The pattern is not confined to these two axes

§4 and §5 each show one axis on which a ranking moves with the measurement recipe. If those were the only two, the honest reading would be that we found two awkward special cases. They are not. Assembling the rest of our own record — suites that predate this paper, and a second hardware backend — turns up the same structure on six further axes. We set the catalogue out here because a reader is entitled to ask how often this happens before accepting §8’s recommendations, and because several of these results were sitting unreported in our own artifacts until we audited them for this draft.

The evidence in this section is **weaker than in §4**, and uniformly so: most of it is $n = 1$ at seed 1337 on a single 8 GB laptop GPU, with no confidence intervals anywhere. We label each entry accordingly and we draw no quantitative conclusion from any of them. The claim this section supports is qualitative and, we think, the more important one: *rank reversal under a change of measurement recipe is the common case, not a curiosity of the attention/minGRU pair*.

6.1 Token budget alone reorders a five-arm board

Suite 13 ranked five mixers at 2.048M tokens; suite 14 extended three of them to 8.192M under a byte-identical configuration — same 12L/768d shape, block 512, batch 8, Muon at 6e-4, FineWeb-edu, seed 1337, `mixer_chunk = 32`. Only the budget changed.

arm	2.048M (suite 13)	8.192M (suite 14)
minGRU	5.8374 (1st of 5)	5.1545 (2nd of 3)
Gated DeltaNet	5.9939 (2nd)	—
Mamba-2	6.0404 (3rd)	5.3831 (3rd of 3)
attention	6.0733 (4th)	5.1362 (1st)

arm	2.048M (suite 13)	8.192M (suite 14)
MLA	6.1556 (5th)	—

Attention — the default the other four are proposed as alternatives to — is fourth of five at the short budget and first at the long one. This is the cleanest statement of the paper’s thesis we have, because nothing but the number of tokens differs, and it is a *board* reordering rather than a single crossing. It is also $n = 1$ per arm, and the 8.192M values are `best_val` fields rather than paired snapshots, so it should be read as a demonstration of the phenomenon and not as a measurement of its size. §4’s $n = 5$ replication exists precisely because this one does not carry an interval.

6.2 6.2 The choice of metric reorders the same arms

The five mixers of suite 13 were separately benchmarked for throughput at fixed shape (suite 17, batch 8, context 512, 124M parameters, same GPU):

arm	quality rank (suite 13)	throughput (suite 17)	throughput rank
MLA	5th (6.1556, worst)	9,334.57 tok/s	1st
attention	4th (6.0733)	7,923.37 tok/s	2nd
minGRU	1st (5.8374, best)	6,674.23 tok/s	3rd
Mamba-2	3rd (6.0404)	333.23 tok/s	4th
Gated DeltaNet	2nd (5.9939)	238.18 tok/s	5th

MLA is last on quality and first on throughput; minGRU is first on quality and third on throughput. The two orderings are close to inverted at the top. Neither metric is wrong — they answer different questions — but a screen that reports one and is read as if it ranked the other will select the opposite arm. Note also the scale of the throughput spread: Mamba-2 and Gated DeltaNet are $24\times$ and $33\times$ slower than attention here, which is a difference in kind rather than degree and is what motivated §6.5.

6.3 6.3 Within a single suite, speed rank and quality rank invert

Suite 18 stacked memory-residency optimizations on a 124M attention model and ranked the arms by throughput. Its `best_val` figures run the other way:

arm	peak tok/s	MFU	<code>best_val</code>
<code>gpu_opt_bs32</code>	13.7K (1st)	25.5%	4.9065 (3rd)
<code>gpu_opt_validate</code>	13.6K (2nd)	25.3%	4.8084 (2nd)
<code>gpu_max</code>	11.9K (3rd)	22.1%	4.8001 (1st)

The ordering is exactly reversed. The three arms differ in batch size (16 / 24 / 32) at a fixed step count, so they do not see equal token budgets and this is emphatically *not* a controlled quality comparison — which is the point. The suite was built to answer a systems question, its throughput conclusion is sound, and reading its ladder as a recipe ladder selects the worst of the three models. We flag this one with some embarrassment: `gpu_opt_bs32`’s `best_val` was the single cell absent from that suite’s own results table until this audit, and its absence is what made the inversion invisible for two months.

6.4 6.4 Batch size reorders throughput among mixers

Suites 22 and 26 ran overlapping hybrid arms at batch 96 and batch 32 respectively on the GH200, with `mixer_chunk = 32` in both. Median per-step throughput across five seeds:

arm	batch 96	batch 32
hybrid_mamba10_attn2	43,941 tok/s	17,753 tok/s
hybrid_gdn10_attn2	31,333 tok/s	24,279 tok/s

At batch 96 the Mamba-2 hybrid is $\sim 1.4\times$ faster than the GDN hybrid, with disjoint seed ranges (43,913–43,946 against 31,330–31,383). At batch 32 the ordering reverses. Mamba-2’s chunked scan needs batch parallelism to fill the device; the delta-rule kernel degrades more gracefully. Two caveats matter. First, the quality comparison between these two suites is confounded — batch size and `lr_max_steps` both change — but the *throughput* comparison is not, because the learning-rate schedule cannot affect tokens per second. Second, these suites ran two jobs per GPU (measured concurrency $1.97\times$), so the absolute figures are contended and one `mamba2` seed spans 15,493–38,365 tok/s. We therefore report the reversal and not the magnitudes, and a single-tenant re-measurement is on the list in §8.4.

6.5 6.5 The kernel implementation decides feasibility, not just speed

Suite 19 replaced the $O(T)$ sequential scans for Mamba-2 and Gated DeltaNet with chunk-parallel kernels, verified against the sequential reference for output and input-gradient parity at $1e-5$ with fp32 accumulation and non-divisible-length coverage:

arm	before	after
Mamba-2 @ bs8/ctx512	333 tok/s	3,224 tok/s ($9.7\times$)
Gated DeltaNet @ bs8/ctx512	238 tok/s	482 tok/s, later $\sim 1,600$ tok/s vectorized
Gated DeltaNet @ bs16/ctx1024	OOM	1,100 tok/s @ 4.0 GB

The last row is the one that matters for ranking: a configuration that was *unrunnable* becomes competitive, purely from an implementation change with no effect on the mathematics. Any board compiled before this work would have recorded GDN at that shape as an absence rather than a number. Kernel maturity is a property of the software, not the architecture, yet it silently determines which arms appear on the board at all.

This section also carries the paper’s most consequential bug: the first version of the chunked scan accumulated in bf16 under autocast, and CPU-only tests did not catch the drift. Every SSM quality number downstream would have been wrong. The fix is to force fp32 inside the scan.

6.6 6.6 The hardware backend reorders both quality and throughput

We maintain a second, independent implementation of the same architecture in Rust and Metal on an M5 Pro. Comparing it against the CUDA reference at the same `sota` shape gives a quality crossover in the training horizon:

horizon	Metal-native (f32)	3070 Ti CUDA (bf16)	leader
3k steps	2.0222	1.9944	CUDA
20k, no warmdown	1.9178	1.9944	Metal
20k + warmdown, seed 1337	1.8969	1.9944	Metal
20k + warmdown, seed 42	1.8876	~1.9860	Metal
100k WSD	1.8828	—	—

The backends swap places somewhere between 3k and 20k steps. This comparison is confounded in several ways we cannot remove — different precision (f32 vs bf16), different flash-attention implementations, and no matched CUDA arm at 20k or 100k — so we present it as a demonstration that “which implementation is better” is horizon-dependent, not as a measurement of either backend.

The backend also reorders the mixers themselves. On CUDA at 124M (suite 17) attention is $1.19\times$ faster than minGRU; on Metal at **sota** scale, measured over 40 FineWeb steps with real kernels, minGRU is **$2.17\times$ faster than attention** (144,367 vs 66,382 tok/s), and Mamba-2 closes from $24\times$ slower than attention to $2.0\times$ slower (33,321 tok/s). The mixer throughput ranking is a property of the kernel library available on the device.

Most striking, the Metal track independently reproduces §4’s headline shape. At 3k steps with matched seeded initialization, minGRU leads attention 2.085016 to 2.123010 (and 2.076374 on a second seed); at 20k steps attention leads 1.89688 to 1.993295. Bias early, capacity late — the same direction as §4, on a different backend, precision, kernel stack and data pipeline. We report this as suggestive rather than as a replication, because the 20k attention arm uses `-/-golden-init` while the minGRU arm is seeded (the golden banks contain no `mingru_*` tensors), so the crossing itself is confounded with initialization. One seeded attention run at 20k would close that gap, and it is the cheapest experiment named anywhere in this paper.

6.7 6.7 What this catalogue is and is not

Counting §4, §5 and this section, we have observed a ranking change places under a change of: token budget, learning-rate horizon, batch size, model scale, training horizon at fixed scale, learning rate, evaluation metric, kernel implementation, and hardware backend. In no case did the architectures, the optimizers or the data change.

We are **not** claiming that all rankings are arbitrary, that these effects are of comparable size, or that any particular arm is secretly better. Several entries above are single-seed observations on one laptop GPU and would not survive a careful replication; §4 exists because we know the difference. What the catalogue establishes is narrower and sufficient for §8: the recipe-dependence in §4 is not an anomaly of one architecture pair, so a screening protocol that does not pin and report its recipe is not measuring a property of the methods it claims to rank. The practices in §8 follow from the pattern, not from any single row above.

7 7. Threats to validity and limitations

7.1 7.1 What these results do not establish

- **No universal crossing token.** The claim is the opposite. Nothing here supports “attention overtakes minGRU at X tokens” for any X outside the exact recipe measured.
- **No 3070 Ti vs GH200 quality ranking.** The two boxes differ in batch size, evaluation count and kernel versions simultaneously. Absolute losses differ by ~ 0.18 – 0.3 nats at matched token markers and the tables are never stacked.

- **No throughput claim.** Every number in §4 is held-out cross-entropy. Quality and tokens-per-second are separate outcomes.
- **No mechanism.** “Bias early, capacity late” fits the curves and does not predict the crossing token, which is what moved. It remains an explanation, not a theorem.
- **Suite 26 does not rerun attention or minGRU.** Its top two rows are suite 22’s sample.
- **Suite 25 cannot locate a late crossing.** Its horizon ends before any batch-32 recipe crosses.

7.2 7.2 Known limitations

- **Single data distribution, single model shape.** Everything is FineWeb-Edu at 12L/768d, sequence 512. Whether the schedule sensitivity in §4.3 persists at other widths, depths or sequence lengths is untested.
- **No μ P, and a single global learning rate.** Runs use standard parametrization with Muon at a fixed 6e-4 across every arm, batch size and schedule. This is the largest uncontrolled factor in the paper and the one most capable of offering a competing explanation for §4.3 and §4.4; it is treated at length in §8, together with the 2×2 follow-up that would settle it and the pre-registered reading of each outcome.
- **Two seeds on the optimizer axis.** §5’s stage-3 and stage-5 intervals are two-seed intervals and are, correctly computed, wide. The rank *reversal* is robust (a 0.53 BPB swing at 128M is far outside any plausible two-seed noise band); the *ordering within the top four* is not.
- **compile = False throughout suites 22–26.** Forced by an Inductor stall on GH200 aarch64. Uniform across arms, but a compiled recipe is a different recipe and might move the crossings again.
- **Recipe drift in suite 22’s ten-arm board.** Documented in §4.5 and repaired by suite 26; the original board should be read only as “what finished.”
- **A kernel bug mid-grid.** Gated DeltaNet produced NaN gradients at finite loss (batch 96 step 39; batch 32 step 55) because `gdn_chunked` exponentiated the full $C \times C$ log-ratio grid and `j > t` entries overflowed, with `.tril()` masking it in the forward pass. Fixed by taking the lower triangle before the exponential, clamping log-ratios at max 0, and clamping alpha. All GDN rows in this paper are post-fix. This was a kernel defect, not a mixer-quality result.
- **A number we reconciled, and the wrong reconciliation we tried first.** The champion run’s final EMA sliding BPB is **2.010659** (seed 1337) in `research/champion-run.json`, while other workspace documents recorded **2.015756**. We first concluded that 2.015756 was a $57 \leftrightarrow 75$ transposition of the audit-7 artifact’s 2.015576, on the reasoning that it matched no artifact on disk. That was wrong, and acting on it would have deleted a real measurement from four documents. There were **three** champion runs, not two: `DECISIONS.md` M15, written 2026-07-19 before Audit 7 existed, independently records the champion at **2.0158** — which 2.015756 rounds to and 2.015576 does not. 2.015756 is the pre-Audit-7 run’s own value; its artifact was superseded and deleted, which makes the figure *unverifiable*, not mistyped. “Matches no artifact on disk” and “was fabricated” are different claims, and we conflated them. The citable value is unchanged at 2.010659, and no comparison in this paper uses either figure. `champion-run.json` additionally carries `"locked": false` — see §8.3 for why it stays that way.

7.3 7.3 Claims withdrawn

Stating these explicitly is part of the result.

1. **“The ~7M attention/minGRU crossover, replicated.”** Withdrawn as a general claim. Suite 14 stands as a valid RTX 3070 Ti / batch 8 / one-seed observation and nothing more.
2. **“Metal trainer BPB 2.038 vs CUDA 1.994 confirms numerical correctness.”** Withdrawn. The two figures come from different flash implementations and the comparison to the 1.9944 CUDA reference is cross-scale, not matched-shape; the same-shape CUDA 128M reference is still unrun. Numerical correctness of the Metal trainer rests on what was actually done — exact checkpoint-and-replay to 1e-6 (§5.2), 57/57 `cargo\test\-\/-release\-\/-lib` gates covering the native optimizer, checkpoint, telemetry and NaN paths, and an independent MLX oracle for the manual GEMM attention backward — not on that comparison. The claim was attached to the wrong evidence. (An “29/29 oracle tests” figure circulated in earlier internal write-ups. A previous version of this section stated that we could not locate its source; that was wrong. It is recorded at `experiment-notes/arch-metal/51-m5-128m-optimizer-funnel-preflight.md`, line 291: “Python correctness/orchestration oracle: **29/29** tests pass (includes induced-swap unlock rejection).” It is a Python orchestration oracle, not a numerical-parity gate, and it is distinct from the 57/57 `cargo` gates above.)
3. **Mixer throughput of ~238,000 tok/s (Mamba-2) and ~240,000 tok/s (minGRU).** Withdrawn as measurement artifacts: the training graph ran the MLP and stem while the mixer path was a zero-tensor stub with flash attention skipped, so the figure measured everything except the thing being compared. Re-baselined honestly on FineWeb at sequence length 256, the same three arms are **144,367 tok/s** (minGRU), **66,382** (attention) and **33,321** (Mamba-2). minGRU is genuinely fast; it is not 240,000-fast, and Mamba-2 at this sequence length is *slower* than attention rather than several times faster.
4. **“Stage-5 intervals do not overlap.”** Withdrawn and replaced by §5.3.
5. **A documented ~4× headroom claim in the quantized GEMV inference kernels.** Withdrawn. Measured properly, those kernels already run at 62–100% of the ~273 GB/s memory-bandwidth ceiling; the 4× came from a microbenchmark that re-uploaded weights on every call, measuring an upload that does not happen in production. About 77% of a quiet decode token is dispatch and barrier overhead, not arithmetic. The kernels were not the bottleneck; the number of kernel launches was.

7.4 7.4 Corrections found by auditing our own record

The claims in §7.3 were withdrawn as we discovered them, over months. Preparing this draft we did something different and more uncomfortable: a systematic audit of every number in our own suite notes, lock files and ledgers against the raw run directories they were derived from. It found nine defects. We record them here rather than silently fixing them, because a paper arguing that measurement protocol determines conclusions has no standing to hide its own protocol failures — and because the *kind* of error is instructive. None of these was a bad measurement. Every one was a correct measurement that was then mis-recorded, partially reported, or lost.

1. **A results table omitted the one cell that contradicted its ranking.** Suite 18 ranked three arms by throughput and reported `best_val` for two of them. The missing third, `gpu_opt_bs32` at **4.9065**, is the fastest arm and the worst model, and its absence is the only reason the speed/quality inversion in §6.3 went unnoticed for two months.
2. **A headline ranking depended on undocumented restarts.** `run128m_20k` is not one run: its `metrics.jsonl` holds **nine start and eight done events**, with per-segment `best_val` from 3.6279 down to 3.5806 and back to 3.6109. The global minimum, **3.5806**, is *better* than the 10k-step run’s 3.621 that we declared the winner — so that suite’s ranking inverts

depending on which number a reader takes, and we reported only the reading that supported our conclusion without knowing we were choosing. Token accounting for that run is also wrong: every `done` records 98,304,000 tokens for a run that reached step 19,990, where the correct figure is 655,360,000. **We withdraw any horizon claim from that suite in either direction.**

3. **We reported a source as unlocatable while it sat in the repository.** §7.3 item 2 previously stated that we could not find the artifact behind a “29/29 oracle tests” figure. It is at `experiment-notes/arch-metal/51-m5-128m-optimizer-funnel-preflight.md`, line 291.
4. **A ledger lost three completed jobs, and they were the ones that mattered.** The exact-128M LR spot-check ledger recorded one job as “running” and omitted three others entirely, although all four had finished on the same afternoon. Those three runs are what opened §8.3.
5. **We broke our own reporting rule two sections after stating it.** §3.2 says a `best_val` field is not a paired snapshot and is never reported as a ranking. §4.1 then reported four `best_val` figures as end-of-run comparisons.
6. **Our knowledge base stopped recording results four weeks before we wrote this.** The project’s `MASTER_ARCHITECTURAL_KB.md` changelog ended at 2026-07-20, so every suite in §4 — 120 runs at five seeds — was absent from the document nominally serving as the project’s verified record.
7. **A summary artifact reported one arm at 80% of the horizon of the others.** In `crossover50m`, three of Mamba-2’s five seeds ran on a 49.17M-token evaluation grid and two on a 49.99M one — the only arm anywhere in these suites whose seeds end on different grids. The `summary.json` was generated by an aligner taking the *exact intersection* of the two grids, which for that arm ends at **40.16M tokens**, and the figure it produced was 4.759; on the fixed aligner it is **4.656**. To be precise about scope, since we are auditing ourselves and not only the artifact: **this defect is not in the §4.5 board**. That board’s Mamba-2 row is `crossover50m_matched32`, whose forty runs all end at 49,987,584 tokens, and it reproduces at 4.596 on recomputation. The stale figure reached a *different* rendering of this paper (see item 8). What the defect does touch here is §4.5’s comparison of arm movement against suite 22, which quotes suite-22 values measured at 49,594,368 tokens against suite-26 values at 49,987,584 — a 0.8% horizon difference, now stated in that section.
8. **The source file of this manuscript was destroyed, and the reconstruction introduced a new error while fixing a real one.** A history rewrite of this repository stripped the Markdown source from the working tree and from every commit. The paper was rebuilt from the run artifacts. That rebuild correctly caught defect 7 above and a mis-stated parameter count, and correctly reported a normal-quantile interval bug in its own checking script. It also “corrected” the §4.5 tie between attention and the $10\times\text{minGRU} + 2\times\text{attention}$ hybrid, on the grounds that the 4.232 figure “appears in no surviving run”. That correction is itself wrong, and §4.5 records why: 4.232 is the hybrid in `crossover50m_matched32`, measured at the same 49,987,584-token point as the attention row, while the replacement figure of 4.275 comes from `crossover50m`, where that arm’s seeds end at 49,594,368 — a different horizon. The reconstruction replaced a matched-horizon comparison with a mismatched one, in the same section where it documented horizon mismatch as the defect.

9. **A correction of ours was itself wrong, and a concurrent audit caught it.** We diagnosed the two champion BPB figures in §7.2 as a digit transposition, because 2.015756 matched no artifact on disk. A separate audit of the same repository found DECISIONS.md M15 — written before the audit pass we compared against — independently recording 2.0158, which corroborates the figure as a third run’s real value rather than a typo. Acting on our diagnosis would have deleted a real measurement from four documents. We had reasoned from absence of evidence, on exactly the axis this paper warns about, while writing the section that warns about it.

The pattern across all nine is that **the failure mode is bookkeeping, not experimentation**. Runs were executed carefully, gated, and seeded; they were then summarised into notes, ledgers, tables and — in the last case — an entire replacement manuscript, and the summaries drifted from the artifacts in ways invisible from the summaries alone. Recommendation 3 in §9 (fingerprint every job) addresses the input side of this. It does not address the output side, and on the evidence above the output side is where our failures actually occurred. We now regard “*re-derive every reported number from the raw run directory before publication*” as a required step rather than a diligence nicety, and §10 states which numbers in this paper were and were not produced that way.

8. The tuning confound: an alternative reading, and direct evidence that it bites

Every result in §4 was produced under **standard parametrization** with a single global Muon learning rate of 6e-4, held fixed across every mixer, every batch size and every schedule. This is the largest uncontrolled factor in the paper, and it deserves its own section rather than a bullet in §7.2, because it admits an alternative reading of our central finding that we cannot currently rule out.

The section runs in four steps: the competing explanation (§8.1); what our seed structure does and does not already bound (§8.2); a measured instance of the same defect on our own optimizer axis, where an inherited learning rate cost nearly twice the deciding margin (§8.3); and the arm we specify to settle it (§8.4).

8.1 The alternative explanation we cannot exclude

Our claim is that the crossing token moves with batch size and learning-rate horizon. A competing explanation is narrower and less interesting:

A single global learning rate cannot be simultaneously near-optimal for a 12-layer attention stack and a 12-layer minGRU stack, nor for the same stack at batch 8 and batch 32. What we measured as “the ranking depends on the recipe” may be partly “one arm was closer to its own optimum than the other, and which arm that was changed with the recipe.”

This is not a hypothetical concern. Three independent results make it concrete:

1. **Optimal learning rate is horizon-dependent.** Bjorck et al. show optimal LR falls as the token horizon grows and follows its own scaling law [31]. Our suite 22/24 runs use a 50M-token cosine and our suite 23 run a 20M-token cosine; a single fixed peak LR of 6e-4 cannot be optimal for both, independently of the schedule-shape effect we attribute the 12.34M → 14.58M shift to.

2. **Optimal hyperparameters are batch-dependent in ways beyond the learning rate.** Marek et al. show that scaling to small batches correctly requires holding Adam’s second-moment half-life fixed in tokens, not its decay rate [30]. We changed batch size by $4\times$ between suites 22 and 25 and adjusted nothing.
3. **Parametrization determines how much of this matters.** Kalra and Barkeshli localize μP ’s advantage over standard parametrization under AdamW to maximizing the embedding-layer learning rate, which in standard parametrization acts as a bottleneck that induces instability [35]. Our embedding layer is trained at the same global rate as everything else.

If the competing explanation is right in full, the correct statement of our result weakens to “under-tuned arms reorder unpredictably,” which is a much less useful claim. If it is right only in part — our current expectation — the crossing token still moves, but by less, and the honest error bar on “12.35M” widens beyond the seed-derived interval we report.

8.2 8.2 What the existing evidence does and does not already constrain

Two things partially bound the concern, and we state them without overclaiming.

Sign consistency across seeds does not help here. All five seeds agreeing on the sign of the gap constrains *seed* noise. It says nothing about a systematic mis-tuning that is shared by all five seeds of an arm, which is exactly what a wrong global learning rate would be.

The 6e-4 value is not arbitrary, but it was tuned on neither arm at these recipes. It was inherited from the earlier 3070 Ti suites. That makes it, at best, a value tuned for one architecture at one batch size at one horizon — precisely the transfer that [31] and [30] say should not be assumed.

8.3 8.3 The concern is not hypothetical: the ordering it hides crosses over

The argument in §8.1 was an appeal to the literature. We can do better. Our own optimizer axis contains the failure §8.1 describes, and measuring it properly produced a result stronger and stranger than the concern we set out to test.

The §5 funnel selected `muon_polar_adamw` at a matrix learning rate of **0.05** over `normuon_adamw` at **0.1**, by 0.031226 BPB. Both values were tuned at `arch02-16m` and **neither was re-tuned at 128M**, the scale at which the selection was made. We re-ran both candidates across a matched eight-point learning-rate grid at the *exact* protocol that produced the selection — `arch02-128m`, 1,000 steps, seeds 42, 2026 and 1337, argv byte-identical to the recorded `exact_128m_1000` jobs apart from the output path. **Fifty-two jobs, about 30 GPU-hours.**

matrix LR	<code>muon_polar_adamw</code>	<code>normuon_adamw</code>	gap (Polar - NorMuon)	leader
0.0035	2.138058	2.144220	-0.006162	Polar
0.005	2.125631	2.128408	-0.002778	Polar
0.008	2.125430	2.121334	+0.004097	NorMuon
0.0125	2.139372	2.121866	+0.017505	NorMuon
0.018	2.141161	2.129755	+0.011406	NorMuon
0.025	2.152357	2.135230	+0.017127	NorMuon
0.035	2.158727	2.146234	+0.012493	NorMuon
0.05	2.171450	2.155408	+0.016042	NorMuon

Means over three seeds; gaps paired by seed. **Every row is sign-consistent across all three seeds** — twenty-four paired comparisons, none dissenting from its side.

The ordering crosses over between lr 0.005 and lr 0.008. `muon_polar_adamw` leads at the two lowest learning rates, `normuon_adamw` at the six highest, each unanimously. The mechanism is visible in the table: the two have **offset flat basins** — Polar’s minimum region spans 0.005–0.008, NorMuon’s spans 0.008–0.0125 — so each wins inside its own and loses inside the other’s. Neither basin has a resolvable interior point at $n = 3$; both are flat to within the per-cell seed spread of roughly 0.006.

There is therefore no recipe-independent answer to which of these two optimizers is better at this scale. The question the funnel asked does not have one.

What the funnel did was pick a side of that crossing without knowing it existed. It compared Polar at 0.05 against NorMuon at 0.1 — both far up the high-LR wall, in the region where NorMuon leads. Measured against their own optima at this protocol, the inherited values cost:

<code>muon_polar_adamw</code>	selected 0.05, best 0.008	penalty 0.046020 +/- 0.003175	= 1.47x the margin
<code>normuon_adamw</code>	selected 0.1, best 0.008	penalty 0.080400 (n = 2)	= 2.57x

The Polar figure is a paired three-seed interval that excludes zero. Both inherited learning rates were 6–12 $\times$ too high, unequally so, and the difference between those two penalties exceeds the margin that decided the outcome.

What we claim. The selection recorded in §5 is **retired**: `research/champion-run.json` remains `"locked": false`. We do **not** crown `normuon_adamw`; that would repeat the original error with the sign flipped, since it loses at the low end and at each candidate’s own best cell the gap is $+0.004097 \pm 0.005085$, which spans zero.

Limits. Sign tests carry this result: only three of eight matched cells have an interval excluding zero, because the effects sit at or below the per-cell seed spread. Neither optimum is a located point. This is one protocol — 128M, 1,000 steps, Metal on an M5 Pro. Four cells were re-run against their recorded funnel values as a drift check and moved by -0.0019, +0.0003, -0.0058 and +0.0021, at most 0.26%, far below the effects above.

What this section is really reporting. §4 shows a ranking between two architectures crossing over as a function of *token budget*. This section shows a ranking between two optimizers crossing over as a function of *learning rate* — the same structure, on an axis we had treated as a nuisance parameter rather than a variable. The funnel’s error was not that it tuned badly. It was that it reported a ranking measured at one point on an axis along which the ranking reverses.

The grid history is part of the result and we state it rather than presenting the final design as if it were the plan. This began as twelve jobs to re-tune one learning rate. It became fifty-two, across five rounds, because each round revealed the previous one had compared something unequal: a truncated grid, then a boundary minimum, then one candidate given lower learning rates than the other. Three times we drew a conclusion from two seeds that a third seed overturned. That trajectory is the honest cost of controlling this confound once, on one axis, for two candidates, and it is the strongest argument we can make that the confound is not second-order.

Full grid, per-seed values, drift checks and limits: `research/d7-lr-retune.json`, regenerated by `scripts/d7_analyze.py` from the run ledger rather than transcribed. Runner: `scripts/d7_lr_retune.py`.

8.4 8.4 The follow-up, specified

We name the arm rather than gesture at it, so that it can be run or criticized.

Design. Repeat the suite 24 recipe — 12L/768d, batch 32, `eval_iters = 20`, 50M-token cosine horizon, 20M-token stop, seeds 1337, 42, 100, 2026, 777 — as a 2×2 :

	attention	minGRU
standard parametrization, global LR	suite 24 (already run)	suite 24 (already run)
μP, per-arm LR swept at a narrow proxy width	new	new

with the μ P cells parametrized per [12] and the proxy sweep run at a reduced width, transferring the tuned LR to the 768-dim target. Depth is held fixed at 12 layers throughout, which sidesteps the depthwise limitations Depth- μ P identifies for stacks whose residual blocks are themselves deep [33]. Two secondary arms are worth the marginal cost:

- **A per-layer standard-parametrization prescription** as in Everett et al. [32], since their result is that this can outperform μ P and that all parameterizations admit transfer — if the crossing token is stable under *both* μ P and per-layer SP but not under a global LR, the finding is about tuning quality, not about μ P specifically.
- **An embedding-layer-LR-only ablation** as in Kalra and Barkeshli [35]: raise only the embedding learning rate by a factor of width under standard parametrization. This is the cheapest possible probe, and if it moves the crossing token materially it localizes the confound to one layer.

Pre-registered readouts. The follow-up answers a yes/no question, and we state the decision rule before running it:

Outcome	Reading
Both μ P crossings land within the suite 22/24 per-seed bands (early 1.03–1.09M, late 11.93–12.58M)	Recipe dependence is not a tuning artifact. §4.3’s schedule effect stands as stated.
Crossings move but the ordering of recipes is preserved — 20M cosine still crosses later than 50M cosine	The schedule effect is real; its magnitude in §4.3 is inflated by mis-tuning and should be restated as an upper bound.
μ P crossings collapse onto a single token across schedules	The schedule effect is a tuning artifact. §4.3 must be withdrawn and §4.4’s batch result re-examined.
The batch-8 arm (suite 25) develops a crossing under μ P within 7.4M tokens	The batch effect is a tuning artifact, and suite 14’s original 6.6–7.4M window is partially rehabilitated.

Cost. Twenty jobs at 20M tokens for the main 2×2 , plus a proxy-width LR sweep, plus ten jobs for each secondary ablation. On the GH200 configuration used for suites 23–25 this is roughly the cost of suite 24 plus suite 25 together — a small fraction of the 50-job suite 22 grid, and the highest-value single experiment remaining in this line of work.

Until it is run, every crossing token in this paper should be read as “the crossing token for this architecture pair *at a global $6e-4$ Muon learning rate under standard parametrization*,” and the §9 recommendations should be read as conditional on the same. We consider recommendation 2 (pin the schedule when truncating) robust to the outcome, because it is a statement about comparing a prefix to its parent recipe rather than about either being correctly tuned; we consider the specific figure “2.2M tokens, 18% of the location” provisional.

9 9. Recommendations

These are the rules we now apply, each earned by a specific failure above.

1. **Report the crossing, not just the endpoint.** If two curves cross, the crossing token is a result and belongs in the paper with per-seed values. If they cross twice, say so.
2. **Never truncate a run without pinning the schedule.** Fix the learning-rate horizon to the *reference* budget and stop early, rather than letting the cosine follow the truncated step count. Otherwise the prefix is a different recipe (§4.3). Record `lr_max_steps` alongside `max_steps` in every config, and verify it per job before reading results.
3. **Fingerprint the recipe on every job and verify before aggregating.** Suite 22’s mixed batch / `eval_iters` board would have been read as a bakeoff without that check. At minimum: batch size, evaluation count, max steps, LR horizon, compile flag, and kernel version.
4. **Never table cells from different recipes together.** Even when the token markers match.
5. **Five seeds for a sign, two for nothing.** A two-seed 95% interval carries a $t_1 = 12.706$ multiplier and will almost never separate close candidates. If you only have two seeds, report both values and claim only a consistent ordering.
6. **A short learning-rate screen is not a learning-rate selection.** Screen at a horizon within the same order of magnitude as the target, or accept that you are ranking EMA lag.
7. **A minimum-over-evaluations field is not a ranking.** Compare paired snapshots at matched token counts.
8. **Record systems-blocked arms as exclusions with the blocking reason**, never as silent substitutions or omissions.
9. **Assume no safe direction of extrapolation.** Small-scale screens under-rank the eventual large-scale champion *and* the large-scale champion under-performs at small scale (§5.5). Both directions failed here.

10 10. Reproduction and artifacts

Repository. <https://github.com/bharathvbcr/MachineLearning> (public). Every path below is relative to that repository root, at commit `a83257a` or later. Two directories referenced by these instructions are deliberately not committed because they are build outputs with in-repo generators: `nanolab/out/` (raw per-job run directories) and `Rust_MLKit/arch_02_value_resid/metal-native/golden/fwd/` (parity activations; regenerate with `python3\metal-native/scripts/export_goldens.py\-\-out\metal-native/golden`).

All mixer suites run through one entry point:

```
python -m nanolab.crossover_replicate list
python -m nanolab.crossover_replicate smoke          # 40-step GPU/CPU check
python -m nanolab.crossover_replicate launch --workers 4
python -m nanolab.crossover_replicate isolates        # matched20 -> bs8 -> matched32
python -m nanolab.crossover_replicate status --out <run-dir>
python -m nanolab.crossover_replicate timing --out <run-dir> --json
python -m nanolab.crossover_replicate plot
```

A note on timing. No suite in this paper reports GPU hours, because the trainer computed the elapsed time of every run, printed it, and discarded it — the `done` record carried only `best_val` and `tokens`. That is fixed as of 2026-08-22 (`Logger.done` now persists `elapsed_s`), but it is not

retroactive: the `timing` command back-estimates suites 22–26 from median tok/s and labels those rows `estimated_from_median_tok_s`, separately from `measured`. Treat any GPU-hour figure for suites 22–26 as an estimate.

Per-suite output roots, each containing `metrics.jsonl` + `config.json` per job plus `queue.json`:

Suite	Output root	Job prefix	Jobs
22	<code>nanolab/out/crossover50m/</code>	<code>cx50_</code>	50
23	<code>nanolab/out/crossover20m_locked/</code>	<code>cx20_</code>	10
24	<code>nanolab/out/crossover20m_matched_lr/</code>	<code>cx20h_</code>	10
25	<code>nanolab/out/crossover8m_bs8/</code>	<code>cx8_</code>	10
26	<code>nanolab/out/crossover50m_matched32/</code>	<code>cx32_</code>	40

Locked summary statistics — the numbers in §4 are read from these, not retyped from logs:

- `experiment-notes/nanolab/artifacts/22-suite22_lock.json`
- `experiment-notes/nanolab/artifacts/23-locked20_lock.json`
- `experiment-notes/nanolab/artifacts/24-matched20_lock.json`
- `experiment-notes/nanolab/artifacts/25-bs8_lock.json`
- `experiment-notes/nanolab/artifacts/26-matched32_lock.json`

Figures: `experiment-notes/nanolab/figures/{22-attn-mingru-crossover, 23-locked20-crossover, 24-matched20-crossover, 25-bs8-crossover, 26-matched32-ranking}. {png,svg,pdf}`.

Optimizer funnel: manifest `research/optimizer-study.json` (candidate list, stage definitions, hard gates, required metrics, and the two blocked-candidate reasons); results `research/native-optimizer-funnel.json` (106 jobs); champion record `research/champion-run.json`; exact-resume gates `research/exact-128m-gate-polar.json` and `research/exact-128m-gate.json`.

Full lab notes with per-suite failure logs, evidence grades and interpretation boundaries: `experiment-notes/nanolab/14, 15, 16, 22, 23, 24, 25, 26`.

Non-nanolab sources this paper depends on. An earlier version of this section listed only the `nanolab/` and `research/` paths above, while §5 and §7.3 in fact rest on the Metal/Rust tracks. Those paths are:

Used by	Path
§5.1 funnel design, stage definitions, the 29/29 orchestration oracle	<code>experiment-notes/arch-metal/51-m5-128m-optimizer-funnel-preflight.md</code>
§5.5 downward-transfer table (Polar vs NS5 at sota, 3k, seeds 1337/42)	<code>experiment-notes/arch-metal/53-loop-research-soft-fatiled-2026-07-23.md</code> and its machine-readable board <code>Rust_MLKit/arch_02_value_resid/metal-native/out/loop_research_2026-07-23/RESULTS.json</code>
§7.3 withdrawal 2 (the retracted Metal-vs-CUDA BPB comparison)	<code>experiment-notes/arch-metal/50-*, Rust_MLKit/arch_02_value_resid/metal-native/README.md, metal-native/DECISIONS.md</code>
§7.3 withdrawal 3 (the retracted ~238k/~240k tok/s figures and their re-baseline)	withdrawn values in <code>experiment-notes/arch-metal/52-context-crossover-metal.md</code> ; honest re-baseline in <code>\ldots/53-\ldots</code> , §“mixer honesty”
§7.3 withdrawal 5 (62–100% of ~273 GB/s; ~77% dispatch overhead)	<code>Rust_MLKit/gemma-metal/bench/results/kernel_roofline_finding.json, Rust_MLKit/gemma-metal/docs/bottleneck.md</code>

Used by	Path
§8 LR-transfer evidence (new; see §8.2)	<code>research/champion-run.json</code> → <code>lr_transfer_finding</code> ; raw runs in <code>out/funnel/polar_exact_lr_spot/</code> (not committed — <code>out/</code> is a build output; <code>ledger.json</code> in that directory carries the reconciled record)

Two scope caveats on the re-baselined throughput figures. The 144,367 / 66,382 / 33,321 tok/s values in §7.3 are Metal measurements at sequence length 256 on 0.78M–1.05M-parameter models, over 40 steps. Every quality result in §4 is CUDA at sequence length 512 on 124M-parameter models. “The same three arms” refers to the mixer *type* only; the two sets of numbers share no shape, scale, backend or sequence length and must not be placed in one table.

What the lock files do and do not contain. `experiment-notes/nanolab/artifacts/*_lock.json` carry the crossing points, the 19.677M gap, and the 50M ranking board (`arm`, `mean`, `lo`, `hi`). They do **not** carry the per-marker tables of §4.2–§4.4, their confidence intervals, or the per-seed ranges shown in the §4.5 board. Those are recomputed from the per-job `metrics.jsonl` under `nanolab/out/`, which is not committed (see the note on build outputs above, and gap B3 in `docs/ISSUES_AND_GAPS_2026-08-22.md`). Reproducing the §4 tables therefore requires re-running the suites or obtaining the run directories; reproducing the crossings and the 50M board does not.

Two reproduction caveats, stated rather than buried:

- **Suites 14, 15 and 16 have no preserved replay command.** The closest runner is `python -m nanolab.train`; every recorded field in the relevant `config.json` must be reconstructed before comparing outputs. This is why those suites are cited as observations and never as the basis for a shipped claim.
- **Suite 23’s artifacts are the short-cosine run and were not regenerated.** The runner now sets `CROSSOVER_LR_HORIZON=50000000` so a future launch preserves the long cosine; reproducing suite 23 as recorded requires the old behavior, and a schedule-matched prefix must use a new output directory.

11 Conclusion

We tried to replicate a crossover we had measured ourselves, at five seeds instead of one, on better hardware, and over six times the token budget. The shape survived. The number did not.

The token at which attention overtakes minGRU on this model is 12.35M under a 50M-token cosine, 14.58M under a 20M-token cosine, and does not exist within 7.4M tokens at batch size 8 — and the originally reported 6.6–7.4M window turns out to sit inside a region where minGRU leads by 0.13–0.18 nats on every seed of every batch-32 recipe we ran. On an entirely separate axis, an optimizer that won a two-seed 16M-parameter stage finished last of four at 128M, and the eventual 128M champion loses when run back down to small scale.

The uncomfortable version of the lesson is that the cheap screen cannot be trusted even as a shortlist. The useful version is narrower and actionable: **the ranking is a property of the measurement recipe as much as of the method, so the recipe must be pinned, fingerprinted, reported, and never silently varied between the screen and the run it is screening for.**

11.1 Provenance of this draft

All 38 literature identifiers cited in §1, §2 and §8 were resolved through the ScholarLM / WisDev MCP (`wisdevSearchPapers`, `wisdevPaperLookup`) against arXiv, OpenAlex, Semantic Scholar and Crossref. Every arXiv ID in the reference list was fetched individually and its title, authors, year and abstract confirmed against the claim made about it in the text; the discovery searches that surfaced [29]–[32] and [35] were run in `methodology` research mode. No citation was written from recall, and no claim is attributed to a paper whose abstract was not read.

Every experimental number in §4 and §5 is read from the locked artifacts listed in §10. Suite statistics come from the `*_lock.json` files; optimizer-funnel figures are recomputed directly from `research/native-optimizer-funnel.json`, which is also the source of the interval correction in §5.3.

ScholarDoc’s drafting pipeline was **not** used for the prose. Its retrieval tools were, as described above. The local stdio server initially returned only a section scaffold because the MCP child process was launched without `GOOGLE_CLOUD_PROJECT` and so could not reach the Vertex backend that the orchestrator was already using; that was fixed, and DocGen was then run three times against a healthy backend. All three runs failed to ground: the best grounded on a single source on an unrelated topic (information-retrieval ranking-model distillation, reached by semantic drift on the word “ranking”), and the last grounded on **zero** sources while still emitting a seven-section draft and a numeric review score. None of that output was merged. The tooling defects are recorded in `docs/ISSUES_AND_GAPS_2026-08-22.md` §1.

References

- [1] L. Feng, F. Tung, M. O. Ahmed, Y. Bengio, and H. Hajimirsadegh, “Were RNNs All We Needed?,” *arXiv:2410.01201*, 2024.
- [2] T. Dao and A. Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” *arXiv:2405.21060*, 2024.
- [3] S. Yang, J. Kautz, and A. Hatamizadeh, “Gated Delta Networks: Improving Mamba2 with Delta Rule,” *arXiv:2412.06464*, 2024.
- [4] DeepSeek-AI, “DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model,” *arXiv:2405.04434*, 2024.
- [5] R. Waleffe, W. Byeon, D. Riach, B. Norick, V. Korthikanti, T. Dao, A. Gu, A. Hatamizadeh, S. Singh, D. Narayanan, G. Kulshreshtha, V. Singh, J. Casper, J. Kautz, M. Shoenybi, and B. Catanzaro, “An Empirical Study of Mamba-based Language Models,” *arXiv:2406.07887*, 2024.
- [6] G. Melis, C. Dyer, and P. Blunsom, “On the State of the Art of Evaluation in Neural Language Models,” *arXiv:1707.05589*, 2017.
- [7] J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith, “Show Your Work: Improved Reporting of Experimental Results,” *arXiv:1909.03004*, 2019.
- [8] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep Reinforcement Learning that Matters,” *arXiv:1709.06560*, 2017.
- [9] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.

- [10] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, “Training Compute-Optimal Large Language Models,” *arXiv:2203.15556*, 2022.
- [11] M. Wortsman, P. J. Liu, L. Xiao, K. Everett, A. Alemi, B. Adlam, J. D. Co-Reyes, I. Gur, A. Kumar, R. Novak, J. Pennington, J. Sohl-dickstein, K. Xu, J. Lee, J. Gilmer, and S. Kornblith, “Small-scale proxies for large-scale Transformer training instabilities,” *arXiv:2309.14322*, 2023.
- [12] G. Yang, E. J. Hu, I. Babuschkin, S. Sidor, X. Liu, D. Farhi, N. Ryder, J. Pachocki, W. Chen, and J. Gao, “Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer,” *arXiv:2203.03466*, 2022.
- [13] R. Schaeffer, B. Miranda, and S. Koyejo, “Are Emergent Abilities of Large Language Models a Mirage?,” *arXiv:2304.15004*, 2023.
- [14] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang, “Muon is Scalable for LLM Training,” *arXiv:2502.16982*, 2025.
- [15] N. Amsel, D. Persson, C. Musco, and R. M. Gower, “The Polar Express: Optimal Matrix Sign Methods and Their Application to the Muon Algorithm,” *arXiv:2505.16932*, 2025.
- [16] Z. Li, L. Liu, C. Liang, W. Chen, and T. Zhao, “NorMuon: Making Muon more efficient and scalable,” *arXiv:2510.05491*, 2025.
- [17] J. Li, J. Tan, H. Xu, J. Zhang, Y. Lu, Y. Sun, Y. Xie, and X. Cai, “MONA: Muon Optimizer with Nesterov Acceleration for Scalable Language Model Training,” *arXiv:2605.26842*, 2026.
- [18] K. Lion, F. Hubler, B. Li, A. Orvieto, and N. He, “Muown: Row-Norm Control for Muon Optimization,” *arXiv:2605.10797*, 2026.
- [19] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *arXiv:1711.05101*, 2017.
- [20] X. Chen, C. Liang, D. Huang, E. Real, K. Wang, Y. Liu, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu, and Q. V. Le, “Symbolic Discovery of Optimization Algorithms,” *arXiv:2302.06675*, 2023.
- [21] H. Liu, Z. Li, D. Hall, P. Liang, and T. Ma, “Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training,” *arXiv:2305.14342*, 2023.
- [22] A. Defazio, Xingyu, H. Mehta, K. Mishchenko, A. Khaled, and A. Cutkosky, “The Road Less Scheduled,” *arXiv:2405.15682*, 2024.
- [23] K. Mishchenko and A. Defazio, “Prodigy: An Expediently Adaptive Parameter-Free Learner,” *arXiv:2306.06101*, 2023.
- [24] G. E. Dahl, F. Schneider, Z. Nado, N. Agarwal, C. S. Sastry, P. Hennig, S. Medapati, R. Eschenhagen, P. Kasimbeg, D. Suo, J. Bae, J. Gilmer, A. L. Peirson, B. Khan, R. Anil, M. Rabbat, S. Krishnan, D. Snider, E. Amid, K. Chen, C. J. Maddison, R. Vasudev, M. Badura, A. Garg, and P. Mattson, “Benchmarking Neural Network Training Algorithms,” *arXiv:2306.07179*, 2023.

- [25] G. Penedo, H. Kydlíček, L. Ben allal, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, and T. Wolf, “The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale,” *arXiv:2406.17557*, 2024.
- [26] S. Hooker, “The Hardware Lottery,” *arXiv:2009.06489*, 2020.
- [27] C. J. Shallue, J. Lee, J. Antognini, J. Sohl-Dickstein, R. Frostig, and G. E. Dahl, “Measuring the Effects of Data Parallelism on Neural Network Training,” *arXiv:1811.03600*, 2018.
- [28] S. McCandlish, J. Kaplan, D. Amodei, and OpenAI Dota Team, “An Empirical Model of Large-Batch Training,” *arXiv:1812.06162*, 2018.
- [29] H. Zhang, D. Morwani, N. Vyas, J. Wu, D. Zou, U. Ghai, D. Foster, and S. Kakade, “How Does Critical Batch Size Scale in Pre-training?,” *arXiv:2410.21676*, 2024.
- [30] M. Marek, S. Lotfi, A. Somasundaram, A. G. Wilson, and M. Goldblum, “Small Batch Size Training for Language Models: When Vanilla SGD Works, and Why Gradient Accumulation Is Wasteful,” *arXiv:2507.07101*, 2025.
- [31] J. Bjorck, A. Benhaim, V. Chaudhary, F. Wei, and X. Song, “Scaling Optimal LR Across Token Horizons,” *arXiv:2409.19913*, 2024.
- [32] K. Everett, L. Xiao, M. Wortsman, A. A. Alemi, R. Novak, P. J. Liu, I. Gur, J. Sohl-Dickstein, L. P. Kaelbling, J. Lee, and J. Pennington, “Scaling Exponents Across Parameterizations and Optimizers,” *arXiv:2407.05872*, 2024.
- [33] G. Yang, D. Yu, C. Zhu, and S. Hayou, “Tensor Programs VI: Feature Learning in Infinite-Depth Neural Networks,” *arXiv:2310.02244*, 2023.
- [34] C. Blake, C. Eichenberg, J. Dean, L. Balles, L. Y. Prince, B. Deiseroth, A. F. Cruz-Salinas, C. Luschi, S. Weinbach, and D. Orr, “u- μ P: The Unit-Scaled Maximal Update Parametrization,” *arXiv:2407.17465*, 2024.
- [35] D. S. Kalra and M. Barkeshli, “Quantifying Hyperparameter Transfer and the Importance of Embedding Layer Learning Rate,” *arXiv:2605.21486*, 2026.
- [36] R. Zhao, D. Morwani, D. Brandfonbrener, N. Vyas, and S. Kakade, “Deconstructing What Makes a Good Optimizer for Language Models,” *arXiv:2407.07972*, 2024.
- [37] L. Choshen, Y. Zhang, and J. Andreas, “A Hitchhiker’s Guide to Scaling Law Estimation,” *arXiv:2410.11840*, 2024.
- [38] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” *arXiv:2003.08505*, 2020.